

Горчак Дмитрий, 212. Отчёт: локальный репозиторий git и задача о фигурах на плоскости

Отчёт о выполненной работе: установка git, создание локального репозитория, работа с ним и текущее состояние программы. Скриншоты сняты с окон MSYS2 на моём компьютере.

Сделано:

- 1) git установлен в MSYS2 и настроен, вся работа сохраняется через локальный репозиторий ~/cones;
- 2) готова иерархия классов Point -> Circle -> Cone; фигура создаётся по количеству чисел в строке файла: два числа – точка, три – круг, четыре – конус;
- 3) площадь основания и объём реализованы методами, а не полями; поля закрыты, доступ через get/set;
- 4) объём считается отдельно, по запросу: номер нужной фигуры передаётся программе аргументом командной строки;
- 5) поле фигур нарисовано в gnuplot: вид сверху, карта высот, ландшафт и объёмный рисунок точками по поверхности фигур;
- 6) фигуры складываются в двунаправленный список STL и упорядочиваются по площади основания и по объёму;
- 7) написаны два эссе – по git и по STL.

Уточнение по фигуре: преподаватель подтвердил, что фигура с круглым основанием и высотой – это конус, а не пирамида. Поэтому класс называется Cone, а объём считается по формуле $V = S * h / 3 = \pi * r * r * h / 3$, где S – площадь основания, взятая у базового класса Circle. Для сравнения в классе оставлен метод volume_ngon(n) – объём пирамиды с правильным n-угольником в основании, вписанным в тот же круг: при росте n он стремится к объёму конуса.

1. Установка git в MSYS2

Git поставлен штатным менеджером пакетов MSYS2 командой pacman -S git. На скриншоте – запись в журнале установки, версия пакета, версия самой программы, путь к ней и начало списка команд из встроенной справки.

```
MSYS2 - git install

=== Горчак Дмитрий, 212. Установка git в MSYS2 ===

$ grep -E 'installed (git|mingw-w64-x86_64-gcc) ' /var/log/pacman.log | tail -2
[2026-05-20T00:55:46+0300] [ALPM] installed mingw-w64-x86_64-gcc (15.2.0-13)
[2026-09-17T18:16:00+0300] [ALPM] installed git (2.55.0-1)

$ pacman -Q git
git 2.55.0-1

$ git --version
git version 2.55.0

$ which git
/usr/bin/git

$ git help -a | head -8
See 'git help <command>' to read about a specific subcommand

Main Porcelain Commands
  add                Add file contents to the index
  am                 Apply a series of patches from a mailbox
  archive            Create an archive of files from a named tree
  backfill            Download missing objects in a partial clone
  bisect             Use binary search to find the commit that introduced a bug
```

Скриншот 1. Установка git и проверка: `pacman -Q git`, `git --version`, `git help -a`

2. Локальный репозиторий

Репозиторий создан прямо в папке с задачей, история ведётся по шагам работы: отдельный коммит на каждый класс, на ввод-вывод, на программу и на скрипт gnuplot.

```
$ cd ~/cones
$ git init -b main
$ git config user.name "Gorchak Dmitriy"
$ git config core.autocrlf false
```

Работа над списком из STL велась в отдельной ветке `stl-list` и влита слиянием с ключом `--no-ff`, поэтому в истории видно, что это была отдельная задача. На коммит, соответствующий сроку 19.09, поставлена метка `stage-19-09`.

```
MSYS2 - local repository

=== Горчак Дмитрий, 212. Локальный репозиторий ~/cones ===

$ cd ~/cones && ls
README.md  circle.cpp  cone.h      figures_io.cpp  main.cpp    plot_figures.gp  txt
bin        circle.h    essays      figures_io.h    make_txt.sh  point.cpp
build.sh   cone.cpp    figures.txt  list_demo.cpp  out          point.h

$ git config --list --local | grep -E 'user|autocrlf' | sed 's/\(user.email=\).*\/\1***@***/'
core.autocrlf=false
user.name=Gorchak Dmitry
user.email=***@***

$ git log --oneline --decorate | head -8
77dab7d (HEAD -> main) Плоская структура проекта; фигура по числу чисел в строке (2/3/4), объём по запросу, точки на поверхности и уравнения фигур
a8d16df Эссе переписаны в формате прошлых работ: команда-пример-пояснение и памятка в приложении
cd62534 Отчёт: обновлённые скриншоты (почта в настройках скрыта)
37e7923 Эссе по git и STL, отчёт с рисунками и скриншотами
c16383e Новый конус перенесён в раздел одиночных, ошибочная строка осталась последней
8e78f1c Ещё один конус в исходных данных
43a246a Слияние ветки stl-list: работа со списком из STL
d5efe6d (stl-list) Двухнаправленный список STL: сортировка по площади и по объёму

$ git status
On branch main
nothing to commit, working tree clean
```

Скриншот 2. Состав папки, локальные настройки, история коммитов и чистое рабочее дерево

Все исходники лежат в одной папке, без вложенных каталогов: так их проще найти и отправить. Собранные программы складываются в bin, всё, что создают программа и gnuplot, – в out; обе папки перечислены в .gitignore и в историю не попадают, поэтому git status показывает чистое дерево сразу после сборки и запуска.

3. Рабочий цикл: правка, сравнение, фиксация

Обычная работа с локальным хранилищем выглядит так: изменить файл, посмотреть, что именно изменилось, подготовить изменение к фиксации и зафиксировать его с понятным сообщением. На скриншоте в файл данных добавлен ещё один круг – строка из трёх чисел.

```
MSYS2 - status diff commit

$ echo ' 10.5  6.0  1.2' >> figures.txt

$ git status -s
M essays/screens/02-repository-log.png
M figures.txt

$ git diff
diff --git a/essays/screens/02-repository-log.png b/essays/screens/02-repository-log.png
index 292cac9..83b138a 100644
Binary files a/essays/screens/02-repository-log.png and b/essays/screens/02-repository-log.png differ
diff --git a/figures.txt b/figures.txt
index e11f1e5..2eb35b9 100644
--- a/figures.txt
+++ b/figures.txt
@@ -38,3 +38,4 @@
# строка с ошибкой: одного числа мало
12.0
+ 10.5  6.0  1.2

$ git add figures.txt

$ git commit -m 'Ещё один круг в исходных данных'
[main 72f09da] Ещё один круг в исходных данных
1 file changed, 1 insertion(+)

$ git log --oneline -3
72f09da (HEAD -> main) Ещё один круг в исходных данных
77dab7d Плоская структура проекта; фигура по числу чисел в строке (2/3/4), объём по запросу, точки на по
верхности и уравнения фигур
a8d16df Эссе переписаны в формате прошлых работ: команда-пример-пояснение и памятка в приложении
```

Скриншот 3. git status, git diff, git add, git commit и история после фиксации

Видно главное свойство diff: git показывает не файл целиком, а одну добавленную строку с отметкой + и несколько строк вокруг неё для понимания места.

4. Сборка, запуск и построение рисунков

Программы собираются одной командой `bash build.sh`: классы компилируются вместе с программой, готовые файлы складываются в папку `bin`. После запуска программа создаёт в папке `out` данные для `gnuplot`, а скрипт `plot_figures.gr` строит по ним рисунки.

```
MSYS2 - build and run
$ ./bin/figures figures.txt 9 | tail -24
18                                total:      156.45      286.89

--- Figure asked on the command line ---
figure 9: Cone(x =  9.00, y =  8.00, r =  3.00, h = 10.00, volume =  94.25)
base area = 28.27
volume    = 94.25
volume of a pyramid with the same circle around its base:
  4-gon   = 60.00
  8-gon   = 84.85
 64-gon   = 94.10

--- Field bounds ---
x: [-1.48; 22.88]
y: [-0.78; 21.18]

--- Data for gnuplot (folder 'out') ---
plan_points.txt, plan_circles.txt, plan_cones.txt -- top view
field.txt -- landscape grid 121 x 121
range.gp -- field bounds for gnuplot
surface_points.txt -- points on the surface of every figure
equations.txt -- equation of every figure

Run 'gnuplot plot_figures.gp' to draw the field.
Program finished.

$ gnuplot plot_figures.gp
plots are ready: out/plan.png, out/map.png, out/landscape.png, out/points3d.png

$ ls -l out
equations.txt
field.txt
landscape.png
map.png
plan.png
plan_circles.txt
plan_cones.txt
plan_points.txt
points3d.png
range.gp
surface_points.txt
```

Скриншот 4. Объём фигуры по запросу, файлы с данными и построение рисунков

Вторым аргументом программе передан номер фигуры – для неё печатаются площадь основания, объём и, для сравнения, объём пирамиды с правильным n -угольником в основании. Границы поля по x и y считает сама программа: она находит крайние точки оснований, добавляет запас и записывает готовые команды `gnuplot` в файл `out/range.gp`, который скрипт подключает командой `load`.

5. Результат: поле фигур

Вид сверху: конусы, круги и точки – каждый тип своим стилем, у конусов подписана высота. Видно, что конусы собраны в две группы и несколько одиночных фигур – это заготовка для будущей задачи о кластерах.

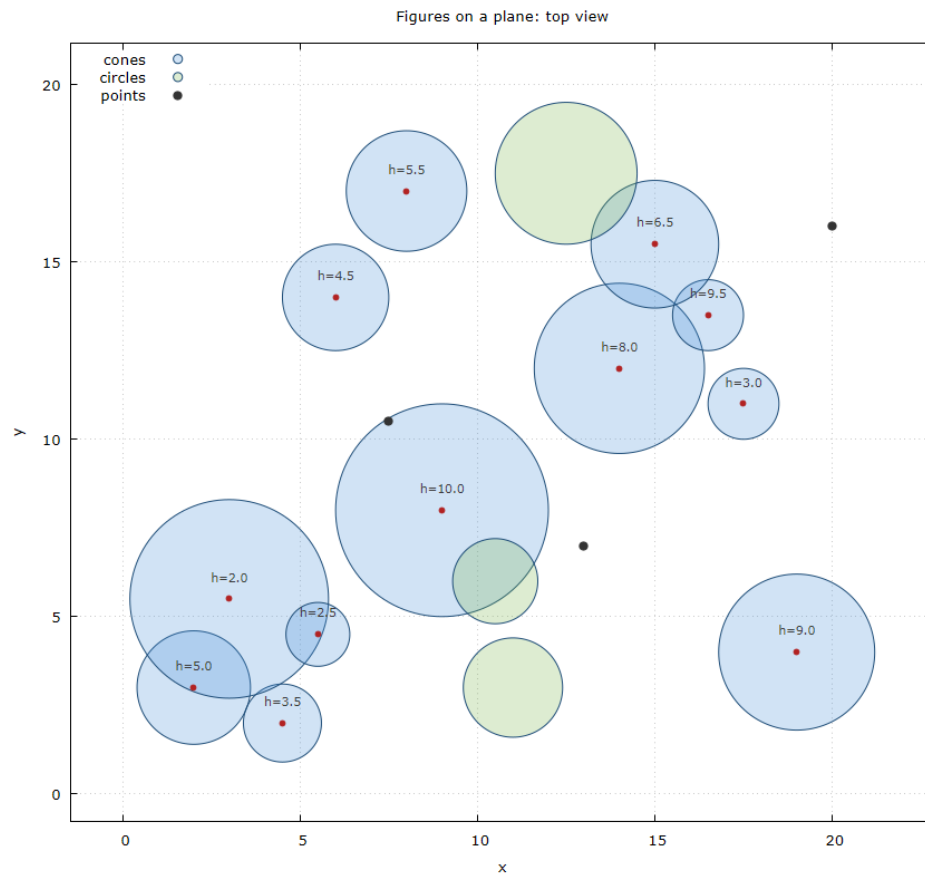


Рисунок 1. Вид сверху: конусы, круги и точки на плоскости

Ландшафт: высота поверхности в точке равна максимуму по всем конусам, поэтому там, где основания пересекаются, виден более высокий конус. Сетку 121 x 121 узел считает программа и сохраняет в файл, gnuplot только рисует.

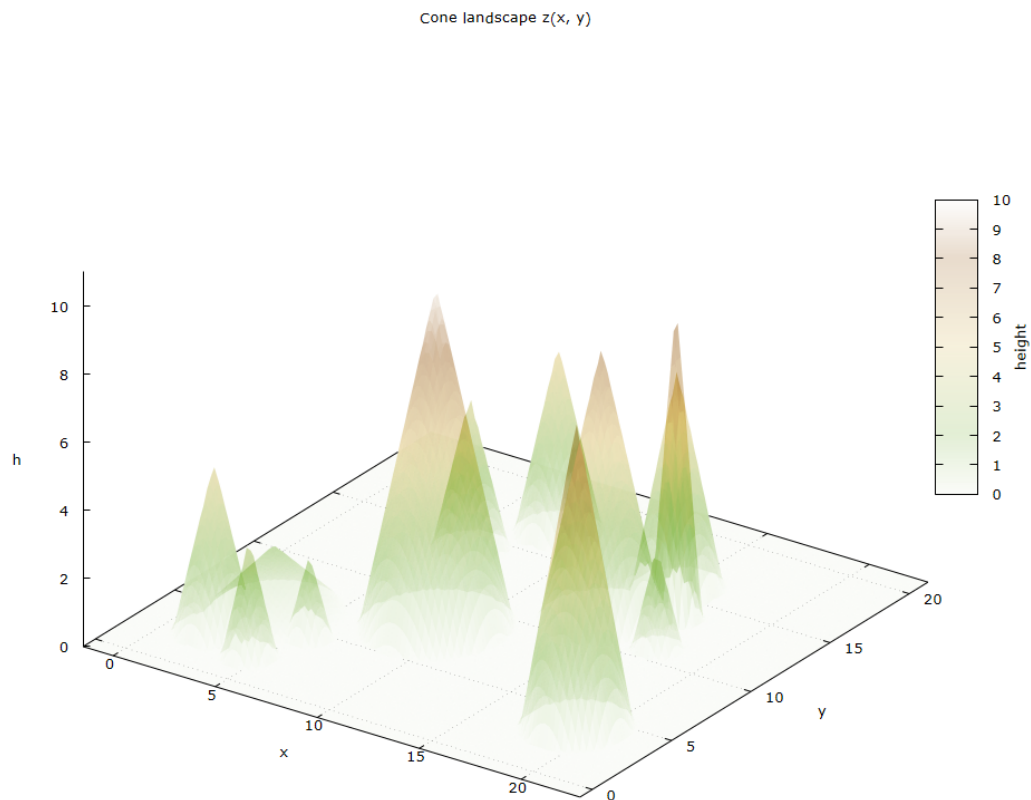


Рисунок 2. Ландшафт $z(x, y)$: поверхность конусов над плоскостью

Третий рисунок показывает фигуры точками на их поверхности. Точки расставлены не перебором пространства, а по уравнению поверхности: на высоте z радиус сечения конуса равен $r * (1 - z / h)$, и по этой окружности точки ставятся через равные углы. Уравнение каждой фигуры программа дополнительно записывает в файл `out/equations.txt`.

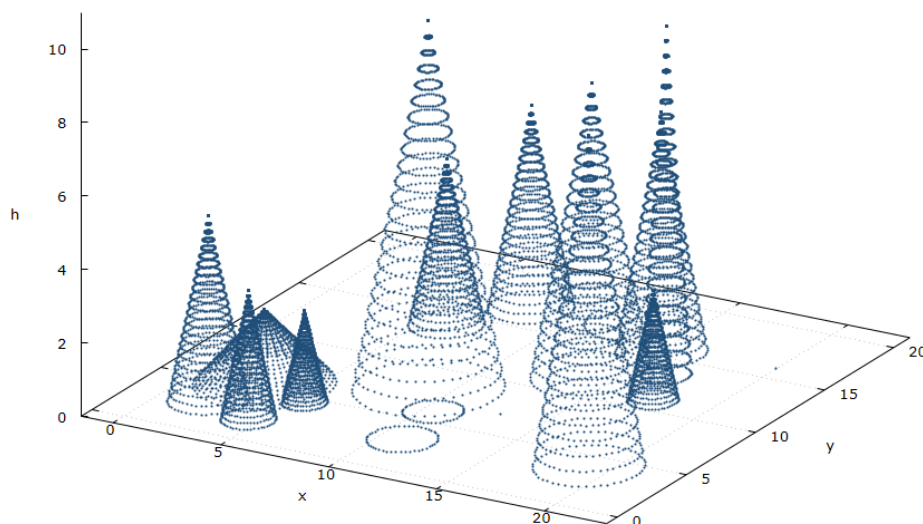


Рисунок 3. Фигуры, нарисованные точками по поверхности

6. Состав репозитория

Файл	Что внутри
point.h, point.cpp	класс Point: координаты, расстояние между точками
circle.h, circle.cpp	класс Circle: радиус и площадь как метод
cone.h, cone.cpp	класс Cone: высота, объём, высота поверхности
figures_io.h, figures_io.cpp	чтение файла, таблица, данные для gnuplot
main.cpp	программа figures: таблица фигур и подготовка рисунков
list_demo.cpp	программа figures_list: двунаправленный список STL
figures.txt	исходные данные: от двух до четырёх чисел в строке
plot_figures.gp	скрипт gnuplot: четыре рисунка
build.sh, make_txt.sh	сборка и копии исходников в .txt
README.md	описание задачи, состава и порядка запуска
essays/	эссе, этот отчёт, скриншоты и рисунки
txt/	копии всех исходников с расширением .txt

7. Что дальше

- 1) выложить репозиторий в облако и прислать ссылку: локальная история переносится командами `git remote add` и `git push`, сами команды работы не меняются;
- 2) генерация точек: у каждой точки, кроме собственных координат, будут две целочисленные координаты её блока сетки;

- 3) дальше – кластеры: матрица попарных расстояний, порог, двоичная матрица смежности и поиск связных компонент волновым алгоритмом. Метод `Point::distance_to` для первого шага уже готов.